

Just like with Python's built-in data types, we can then use the variable `oLightSwitch` to call the methods available in the `oLightSwitch` class.

Definition of an Object

To summarize this section, I'll give my formal definition of an *object*.

object

Data, plus code that acts on that data, over time.

A class defines what an object will look like when you instantiate one. An object is a set of instance variables and the code of the methods in the class from which the object was instantiated. Any number of objects can be instantiated from a class, and each has its own set of instance variables. When you call a method of an object, the method runs and uses the set of instance variables in that object.

Building a Slightly More Complicated Class

Let's build on the concepts introduced so far and work through a second, slightly more complicated example in which we'll make a dimmer switch class. A dimmer switch has an on/off switch, but it also has a multiposition slider that affects the brightness of the light.

The slider can move through a range of brightness values. To make things straightforward, our dimmer digital slider has 11 positions, from 0 (completely off) through 10 (completely on). To raise or lower the brightness of the bulb to the maximum extent, you must move the slider through every possible setting.

This `DimmerSwitch` class has more functionality than our `LightSwitch` class and needs to remember more data:

- The switch state (on or off)
- Brightness level (0 to 10)